

Distributed Systems & Middleware Technologies

Entertainment Ticket Reservation System

Van Khai Do

Mohammed Ragab Elsinousi Abdalhady

Introduction

Problem context

- Modern ticketing systems must handle thousands of concurrent reservation requests
- **Risks:** overselling, double booking, timing violations
- **Requires:** strong synchronization, fault handling, and consistency guarantees

Objectives

This project aims to build a system with the following design goals:

- Correct seat-level synchronization under contention
- Automatic timeout and failure handling
- Asynchronous, interoperable Java–Erlang architecture
- Distributed, fault-tolerant design
- Clean REST API and testability

Synchronization Issues

- **Seat-Level Mutual Exclusion**

Concurrent hold requests must guarantee only one successful hold per seat at any time.

- **State Consistency & Race Conditions**

The system must prevent invalid transitions caused by overlapping holds, confirmations, and timeouts, enforcing a strict state machine:

free → *held* → *sold* or *free* → *held* → *free* (timeout).

- **Multi-Node Coordination**

In a distributed Erlang setup, all operations for a given seat must be routed deterministically to the same node to preserve atomicity.

- **Java–Erlang Request/Response Matching**

Since Java HTTP requests are synchronous and Erlang messaging is asynchronous, the gateway must use correlation mechanisms to correctly pair requests with responses.

Communication Issues

- **Java ↔ Erlang Protocol**

Java and Erlang run on different runtimes; a well-defined message protocol via JInterface prevents misinterpretation and incompatibility.

- **Asynchronous Notifications**

Timeout and confirmation events are decoupled from seat management using Kafka, ensuring the critical reservation path remains non-blocking.

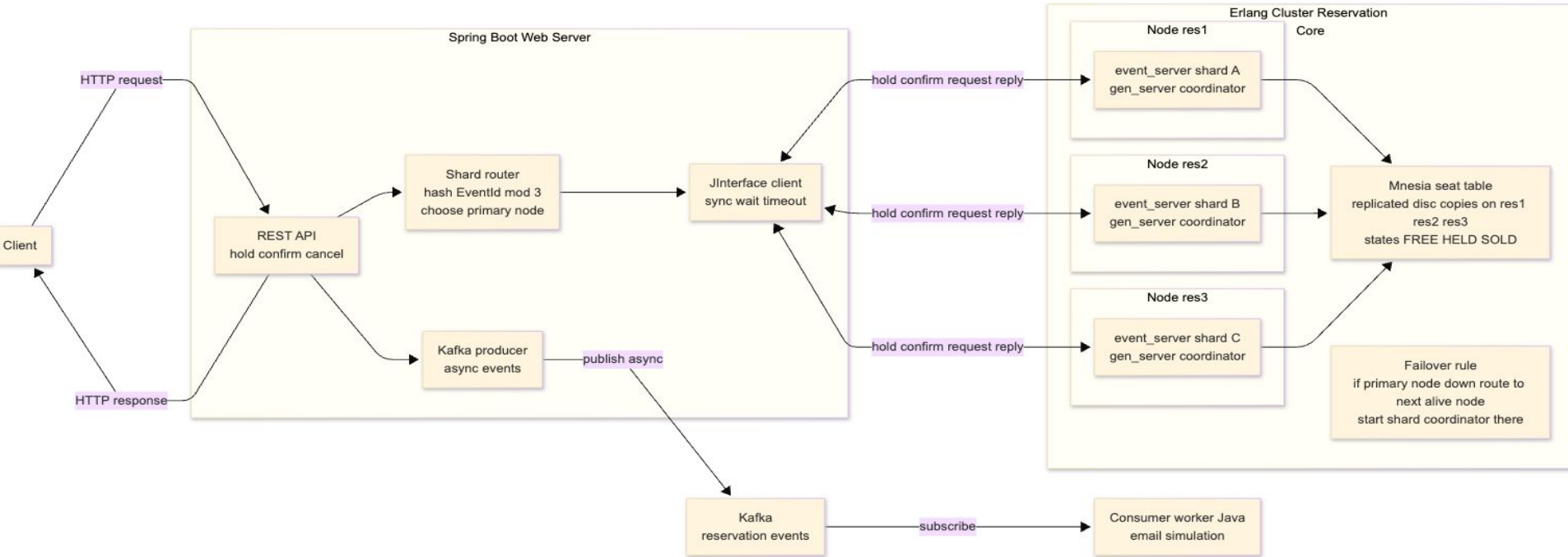
- **Failures and Timeouts**

The system support timeout and failover mechanisms.

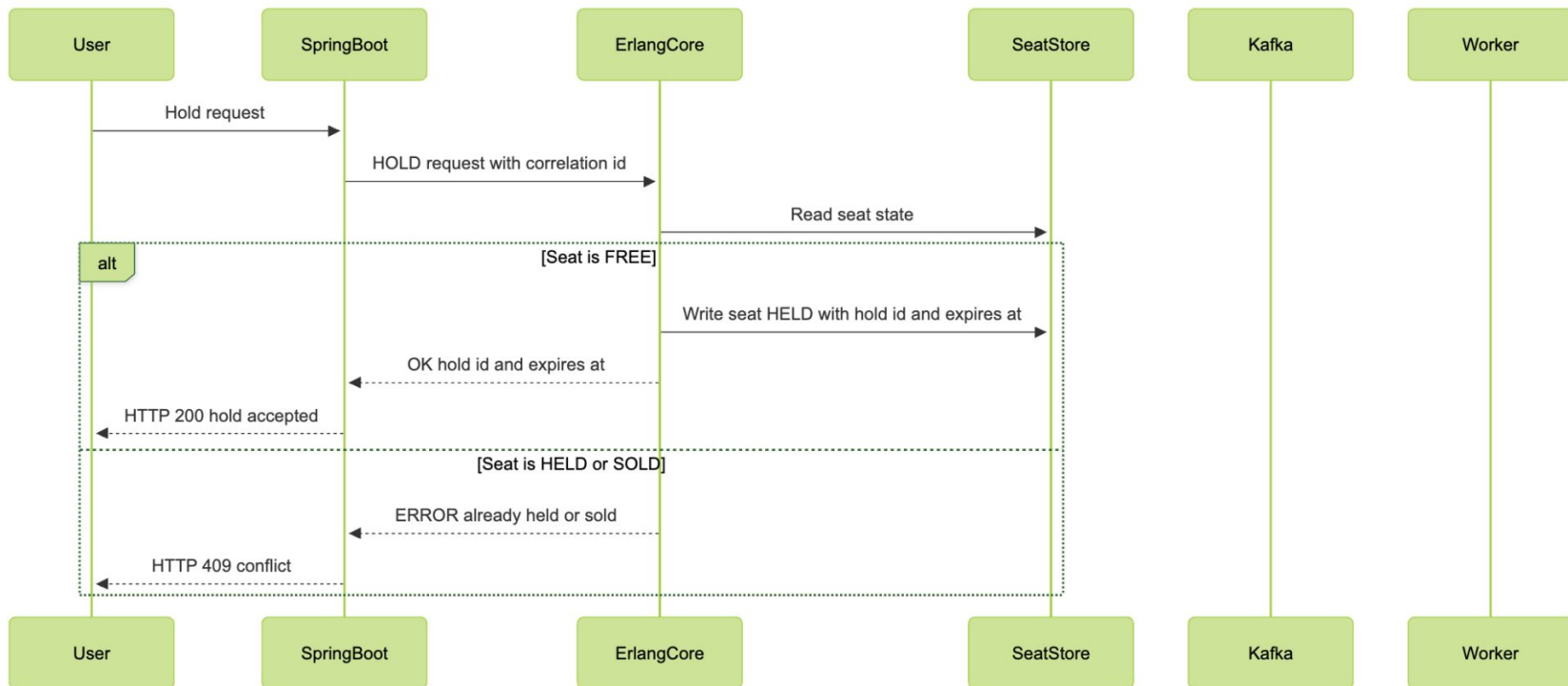
Architecture

- Erlang core with Mnesia (seat state)
- Spring Boot gateway (REST + routing)
- Kafka for asynchronous events
- Worker services for notifications (Java)
- Seat-UI for simulation (React, Typescript)

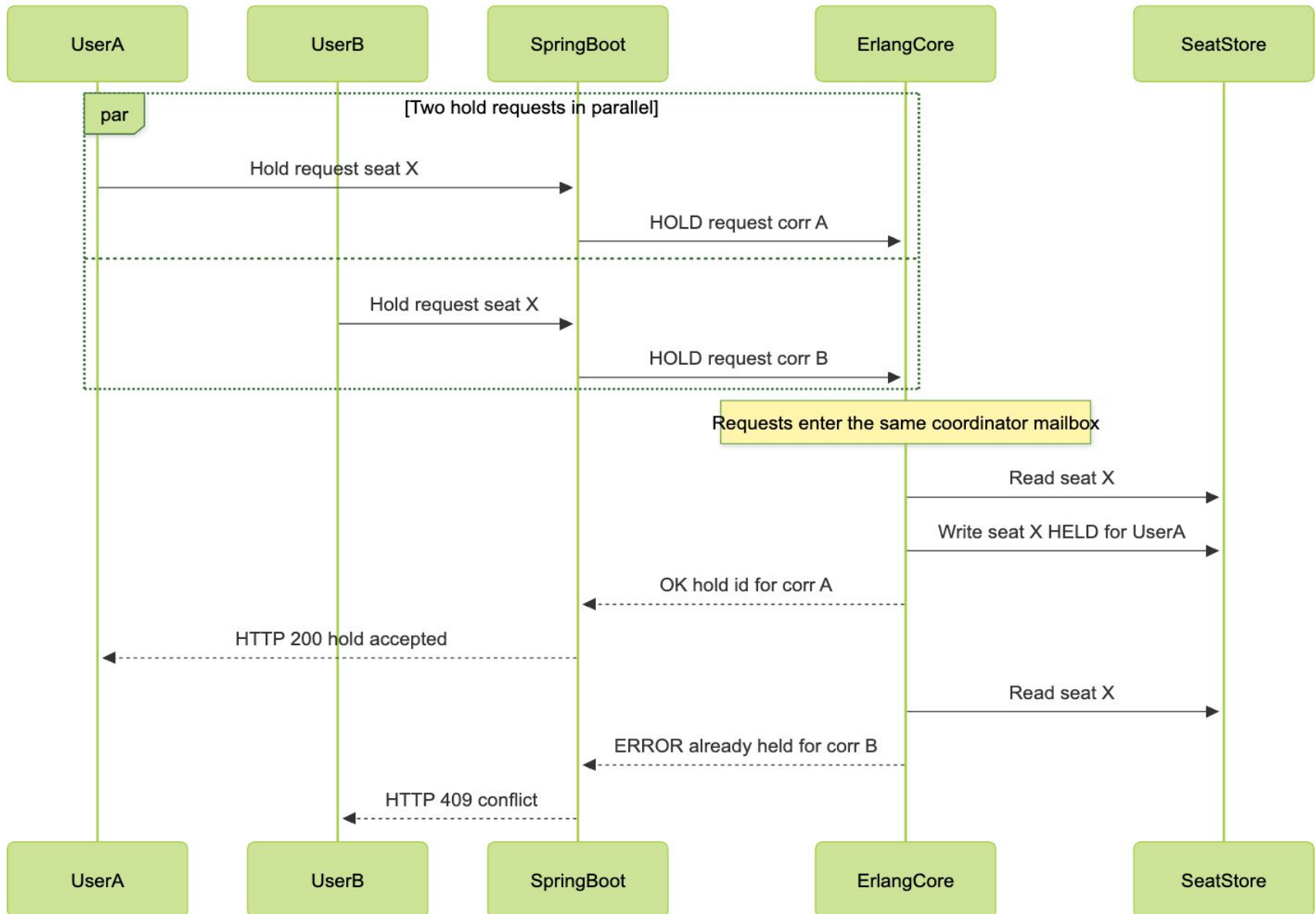
Architecture



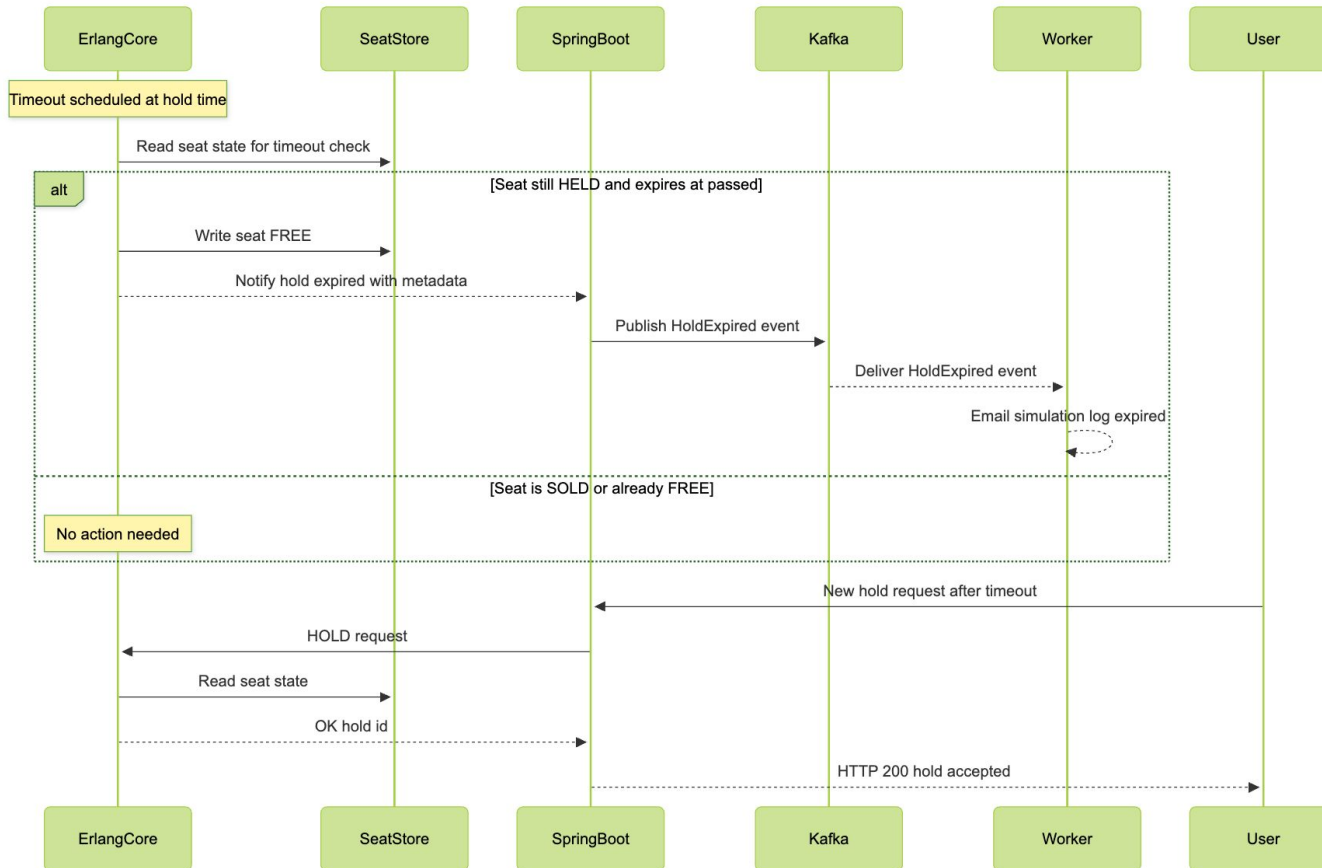
Sequence Diagram - Hold



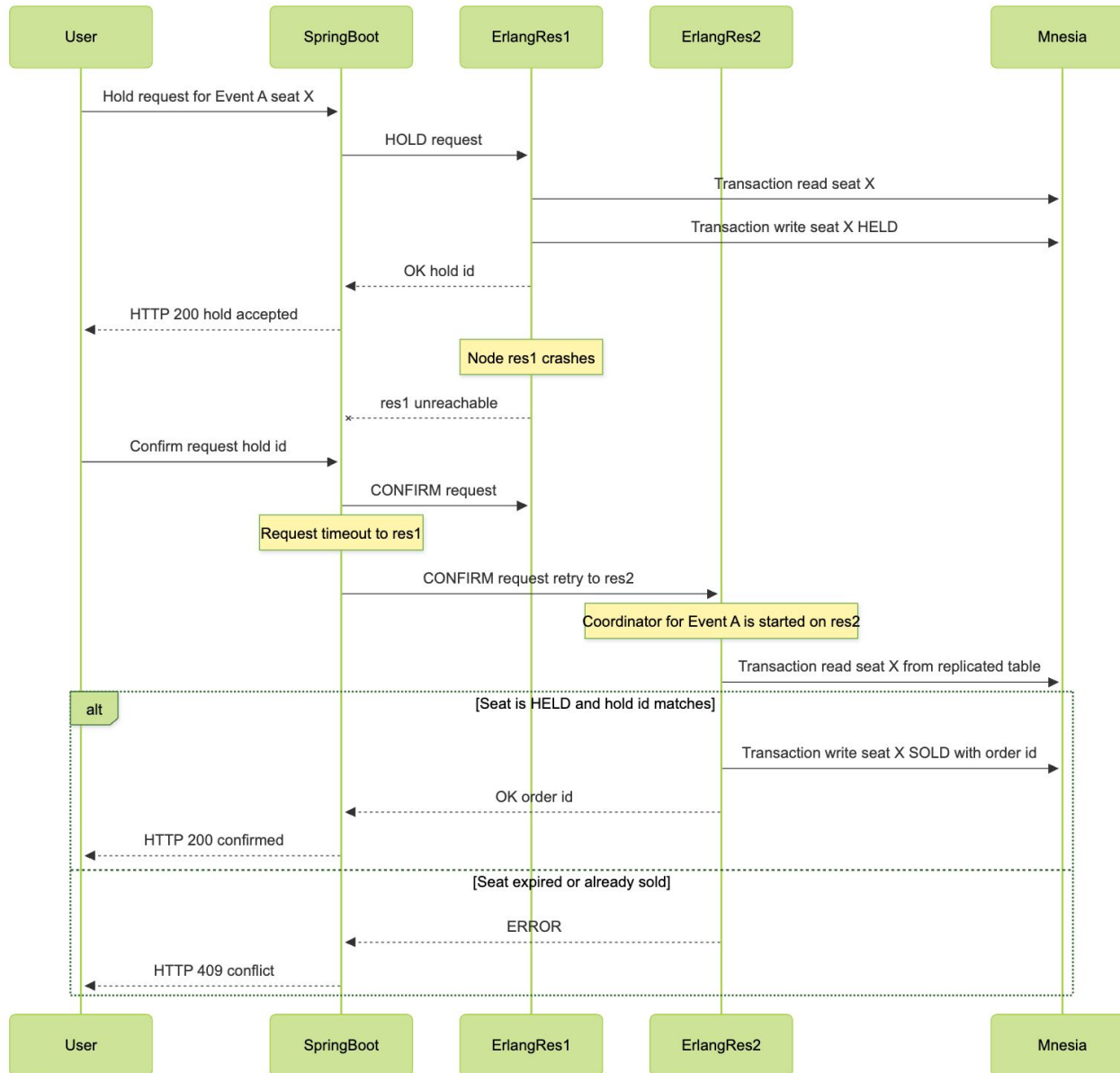
Sequence Diagram - Hold Concurrency



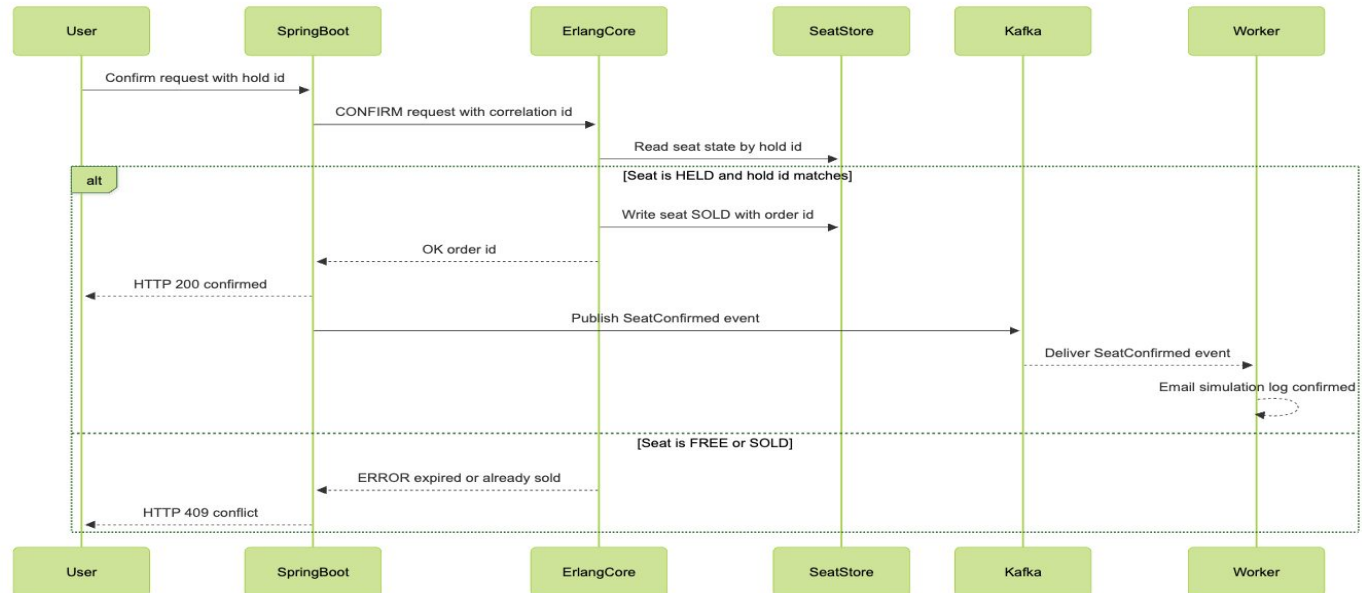
Sequence Diagram - Hold Expired



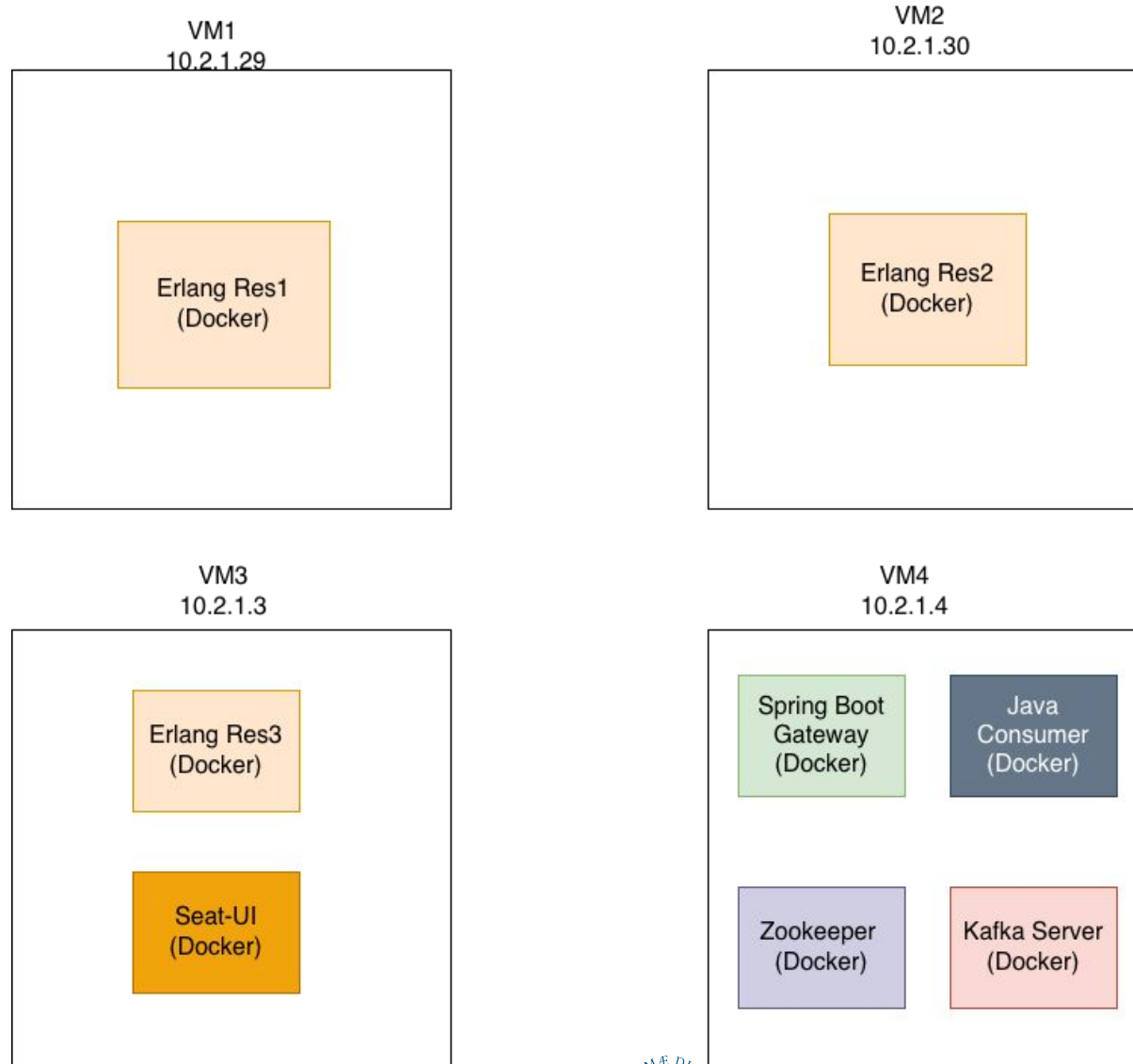
Sequence Diagram - Failover



Sequence Diagram - Confirm



Deployment Strategies



Simulation Seat-UI

← → ↻ ⚠ Not Secure 10.2.1.3:8000

Ticket Reservation System

User ID: user6693

Change

Refresh Seats

☐ Free ☐ On Hold ☒ Confirmed

Zone A

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Zone B

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Zone C

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Zone D

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Zone E

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Zone F

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

Terminal